PROGRAMACION AVANZADA

SOCKET Y HILOS

Este proyecto consiste en el desarrollo de un sistema cliente-servidor en Java que permite la conexión simultánea de múltiples usuarios. El objetivo principal es demostrar cómo se puede implementar la comunicación en red utilizando sockets, junto con el uso de hilos para lograr concurrencia, es decir, que varios clientes puedan interactuar con el servidor al mismo tiempo sin interferirse entre sí.

El sistema está compuesto por dos partes fundamentales: el servidor y el cliente. El servidor es el encargado de esperar conexiones entrantes en un puerto específico, en este caso el puerto 6000. Cuando un cliente se conecta, el servidor establece una comunicación y le solicita su nombre. A partir de ese momento, se crea un hilo independiente para ese cliente, lo que permite que cada conexión sea gestionada de manera individual.

El uso de hilos es clave en este proyecto, ya que permite que el servidor pueda atender múltiples clientes de forma simultánea. Cada cliente tiene asociado un hilo que se encarga de recibir sus solicitudes y enviar respuestas, evitando que una conexión bloquee a las demás.

ServidorHilo.java | Servidor.java

Practica de Socket > src > Servidor.java > ...

```java
import java.io.*;
import java.net.*;
import java.util.*;

public class Servidor {

    // Lista global donde guardamos todos los clientes conectados
    public static List<ServidorHilo> clientes = new ArrayList<>();

    Run | Debug
    public static void main(String[] args) {

        final int PUERTO = 6000;

        try {
            // Se crea el servidor en el puerto indicado
            ServerSocket servidor = new ServerSocket(PUERTO);
            System.out.println("Servidor iniciado...");

            // Bucle infinito para aceptar múltiples clientes
            while (true) {

                // Espera a que un cliente se conecte
                Socket socket = servidor.accept();

                // Flujos de entrada y salida de datos
                DataInputStream in = new DataInputStream(socket.getInputStream());
                DataOutputStream out = new DataOutputStream(socket.getOutputStream());

                // Se le pide el nombre al cliente
                out.writeUTF("Escribe tu nombre:");
                String nombre = in.readUTF();

                // Se crea un hilo para manejar a ese cliente
                ServidorHilo hilo = new ServidorHilo(socket, in, out, nombre);

                // Se guarda en la lista de clientes
                clientes.add(hilo);

                // Se inicia el hilo
                hilo.start();

                System.out.println("Cliente conectado: " + nombre);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

ClienteHilo.java | Cliente.java

Practica de Socket > src > Cliente.java > ...

```java
import java.io.*;
import java.net.Socket;
import java.util.Scanner;

public class Cliente {

    Run | Debug
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        final String HOST = "localhost";
        final int PUERTO = 6000;

        try {
            // Se conecta al servidor
            Socket sc = new Socket(HOST, PUERTO);

            DataInputStream in = new DataInputStream(sc.getInputStream());
            DataOutputStream out = new DataOutputStream(sc.getOutputStream());

            // Recibe mensaje de bienvenida
            String bienvenida = in.readUTF();
            System.out.println(bienvenida);

            // Envía nombre
            String nombre = input.nextLine();
            out.writeUTF(nombre);

            // Se crea el hilo del cliente
            ClienteHilo hilo = new ClienteHilo(in, out);
            hilo.start();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

ClienteHilo.java × Cliente.java

Practica de Socket > src > ClienteHilo.java > ...

```java
import java.io.*;
import java.util.Scanner;

public class ClienteHilo extends Thread {

    private DataInputStream in;
    private DataOutputStream out;

    public ClienteHilo(DataInputStream in, DataOutputStream out) {
        this.in = in;
        this.out = out;
    }

    @Override
    public void run() {

        Scanner sc = new Scanner(System.in);

        boolean salir = false;

        while (!salir) {

            try {
                // Menú de opciones
                System.out.println("\n1: Hora");
                System.out.println("2: Resolver operación");
                System.out.println("3: Ver clientes");
                System.out.println("4: Enviar mensaje");

                int opcion = sc.nextInt();
                sc.nextLine();

                // Enviar opción al servidor
                out.writeInt(opcion);

                switch (opcion) {

                    case 1:
                        System.out.println("Hora: " + in.readUTF());
                        break;

                    case 2:
                        System.out.println("Ingrese operación (ej: RESOLVE \"5+3*2\"):");
                        String op = sc.nextLine();
                        out.writeUTF(op);
                        System.out.println(in.readUTF());
                        break;

                    case 3:
                        System.out.println(in.readUTF());
                        break;

                    case 4:
                        System.out.println("Mensaje:");
                        String msg = sc.nextLine();
                        out.writeUTF(msg);
                        break;
                }

            } catch (IOException e) {
                e.printStackTrace();
                salir = true;
            }
        }
    }
}
```

Herram

Captura de

ServidorHilo.java × Servidor.java

Practica de Socket > src > ServidorHilo.java > ...

```java
import java.io.*;
import java.net.Socket;
import java.util.*;
import java.time.LocalDateTime;

public class ServidorHilo extends Thread {

    private Socket socket;
    private DataInputStream in;
    private DataOutputStream out;
    private String nombreCliente;

    // Constructor
    public ServidorHilo(Socket socket, DataInputStream in, DataOutputStream out, String nombreCliente) {
        this.socket = socket;
        this.in = in;
        this.out = out;
        this.nombreCliente = nombreCliente;
    }

    @Override
    public void run() {

        try {
            // Bucle para escuchar constantemente al cliente
            while (true) {

                // Recibe la opción elegida por el cliente
                int opcion = in.readInt();

                switch (opcion) {

                    case 1:
                        enviarHora();
                        break;

                    case 2:
                        resolverOperacion();
                        break;

                    case 3:
                        listarClientes();
                        break;

                    case 4:
                        enviarMensajeATodos();
                        break;

                    default:
                        out.writeUTF("Opción inválida");
                }
            }

        } catch (IOException e) {
            // Si el cliente se desconecta, lo eliminamos de la lista
            System.out.println(nombreCliente + " se desconectó.");
            Servidor.clientes.remove(this);
        }
    }

    // --------------- FUNCIONES ---------------

    // Envía la fecha y hora actual
    private void enviarHora() throws IOException {
        String hora = LocalDateTime.now().toString();
        out.writeUTF(hora);
    }

    // Recibe una operación matemática y la resuelve
    private void resolverOperacion() throws IOException {...

    // Envía la lista de clientes conectados
    private void listarClientes() throws IOException {...

    // Envía un mensaje a todos los clientes conectados
    private void enviarMensajeATodos() throws IOException {...

    // --------------- CALCULADORA ---------------

    // Procesa el comando RESOLVE
    private static String calculadoraFunciones(String input) {...

    // Resuelve la ecuación respetando prioridad de operadores
    private static double resolverEcuacion(String ecuacion) {...
}
```

```
PROBLEMAS 80   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS   SPELL CHECKER 75

Hora: 2026-04-21T17:06:48.192095800

1: Hora
2: Resolver operación
3: Ver clientes
4: Enviar mensaje
2
Ingrese operación (ej: RESOLVE "5+3*2"):
RESOLVE "5+3*2"
Resultado: 11.0

1: Hora
2: Resolver operación
3: Ver clientes
4: Enviar mensaje
4
Mensaje:
HOLA A TODOS

1: Hora
2: Resolver operación
3: Ver clientes
4: Enviar mensaje
3
joaco: HOLA A TODOS

1: Hora
2: Resolver operación
3: Ver clientes
4: Enviar mensaje
3
Clientes conectados:
- joaco
- MARINO
```

```
PROBLEMAS 80   SALIDA   CONSOLA DE D

PS C:\Users\Usuario\OneDrive\Docu
ages' '-cp' 'C:\Users\Usuario\App
88\bin' 'Servidor'
Servidor iniciado...
Cliente conectado: joaco
Cliente conectado: MARINO
```